

DL26, Assignment 2

Hai Nguyen Hoang (honguyea, 2267317)
Ameya Dinesh Kurme (amekurme, 2262710)

April 26, 2026

Task 2a)

MNIST-1D starts with 10 hand-crafted templates of length 12, one per digit class (0–9). Each template is a 1D signal whose curve shape resembles the corresponding digit when rotated. Templates are whitened and scaled down by a factor of 6.

To generate each example, *make_dataset* selects the appropriate template and applies a *transform* pipeline: (1) **Padding**: 36–60 zeros appended. (2) **Interpolation**: resampled to intermediate length 72. (3) **Scaling**: amplitude randomly scaled by $\pm 20\%$. (4) **Translation**: circular shift up to 48 positions. (5) **Correlated noise**: Gaussian-filtered noise added to zero-padded regions. (6) **IID noise**: small Gaussian noise added everywhere. (7) **Shear**: random linear slope added. (8) **Downsampling**: resampled to final length 40.

After generating 500 examples per class (5000 total), the dataset is shuffled, normalized to zero mean and unit std, and split 80/20 into train (4000) and test (1000).

Task 2c)

Given - $channels(C)=5$, $kernel_size(k)=3$, $stride(s)=2$, $padding(p)=1$.

Conv1d params: $C_{out} \times C_{in} \times k + C_{out}$. Length dimension: $\lfloor (L + 2p - k)/s + 1 \rfloor$.

Conv block 1:

- conv1: Conv1d($1 \rightarrow 5$, $k=3$, $s=2$, $p=1$): $params = 5 \times 1 \times 3 + 5 = 20$, length: $\lfloor (40+2-3)/2+1 \rfloor = 20$
- conv2: Conv1d($5 \rightarrow 5$, $k=3$, $s=2$, $p=1$): $params = 5 \times 5 \times 3 + 5 = 80$, length: $\lfloor (20+2-3)/2+1 \rfloor = 10$
- MaxPool($k=2$, $s=2$): length: $10/2 = 5$

Conv block 2:

- conv3: Conv1d($5 \rightarrow 5$, $k=3$, $s=2$, $p=1$): $params = 5 \times 5 \times 3 + 5 = 80$, length: $\lfloor (5+2-3)/2+1 \rfloor = 3$
- conv4: Conv1d($5 \rightarrow 5$, $k=3$, $s=2$, $p=1$): $params = 5 \times 5 \times 3 + 5 = 80$, length: $\lfloor (3+2-3)/2+1 \rfloor = 2$
- MaxPool($k=2$, $s=2$): length: $2/2 = 1$

Linear layer: After flattening: $5(channels) \times 1(length) = 5$, so $linear_in=5$.
Linear($5 \rightarrow 10$): $10 \times 5 + 10 = 60$.

Total: $20 + 80 + 80 + 80 + 60 = \mathbf{320}$ parameters. Verified with *count_model_params*: 320.

Task 2d): i), ii), and iii)

(i) Why is a map-style dataset more suitable?

A map-style dataset supports random access via `__getitem__`, allowing the *DataLoader* to pick arbitrary indices to form shuffled mini-batches each epoch. An iterable-style dataset only supports sequential iteration, which makes shuffling and random batching difficult without additional logic. For MNIST-1D, all training examples are already loaded in memory as tensors, so random access is efficient. Since we need shuffled mini-batches for stochastic gradient descent, a map-style dataset is more suitable.

(ii) Is TensorDataset map-style or iterable-style?

TensorDataset is a *map-style* dataset. It inherits from *torch.utils.data.Dataset* (not *IterableDataset*) and implements `__getitem__` and `__len__`. Internally, it stores the provided tensors and indexes them and `__getitem__(index)` returns a tuple containing `tensor[index]` for each stored tensor.

(iii) Why CrossEntropyLoss over NLLLoss?

NLLLoss expects its input to already be log-probabilities, meaning we would need to apply *log_softmax* as the last step of the model. *CrossEntropyLoss* combines *log_softmax* and *NLLLoss* into a single operation, so it accepts raw logits directly.

Our *SimpleCNN* outputs raw 10-dimensional scores (logits) from the final linear layer without applying softmax. Therefore, *CrossEntropyLoss* is the correct choice because it handles the softmax internally. Using *NLLLoss* on raw logits would produce incorrect results. Additionally, *CrossEntropyLoss* is numerically more stable because it fuses the log-softmax computation, avoiding potential overflow and underflow.

Task 2e)

Parameter count of SimpleMLP with $Z = 256$:

- lin1: Linear($40 \rightarrow 256$): $40 \times 256 + 256 = 10,496$
- lin2: Linear($256 \rightarrow 256$): $256 \times 256 + 256 = 65,792$
- lin3: Linear($256 \rightarrow 10$): $256 \times 10 + 10 = 2,570$
- **Total:** 78,858 **parameters** (Verified in the notebook)

Conjecture: The MLP has 78,858 parameters—roughly $13\times$ more than the CNN’s 6,060. Despite this, the MLP will likely perform *worse* than the CNN. The CNN exploits local spatial structure through weight sharing and local receptive fields, recognizing patterns regardless of position (translation invariance). The MLP treats each input position independently with no notion of locality or context, so it must learn the same pattern separately for every possible position. More parameters do not compensate for the wrong inductive bias.

Task 2f)

The results confirm the conjecture. Despite having 78,858 parameters ($\sim 13\times$ more than the CNN), the MLP achieves a test accuracy of only 62.9%, compared to the CNN’s 89.4%. This demonstrates that parameter count alone does not determine performance, but the inductive bias matters more. The CNN’s convolutional structure encodes locality as context and translation invariance, which matches the spatial nature of MNIST-1D data. The MLP lacks this spatial awareness and generalizes poorly despite its larger capacity.

Task 2h)

Conjecture: When the spatial dimension is shuffled, the local structure and positional relationships between neighboring points are destroyed. The CNN should perform *much worse*, because its advantage relies entirely on local convolutional filters detecting patterns in neighboring positions and with shuffling, neighbors are meaningless. The MLP should be *unaffected*, since it already treats each position independently via fully connected weights and never relied on spatial locality. The MLP might slightly outperform the CNN on shuffled data.

Task 2i)

The results confirm the conjecture. On the shuffled dataset, the CNN's test accuracy drops from 89.4% to 57.7% (a decrease of over 30 percentage points), while the MLP's performance remains essentially unchanged at 63.2% (compared to 62.9% on the original data).

This validates that the CNN's advantage on the original data was entirely due to its spatial inductive bias. Convolutional filters exploit local patterns and translation invariance, which become a liability when spatial structure is destroyed by shuffling. The MLP, having no assumptions about spatial structure, is indifferent to shuffling.

Notably, the MLP now *outperforms* the CNN (63.2% vs. 57.7%), confirming that without spatial structure to exploit, the CNN's architecture (local receptive fields, weight sharing) becomes a disadvantage rather than a strength.

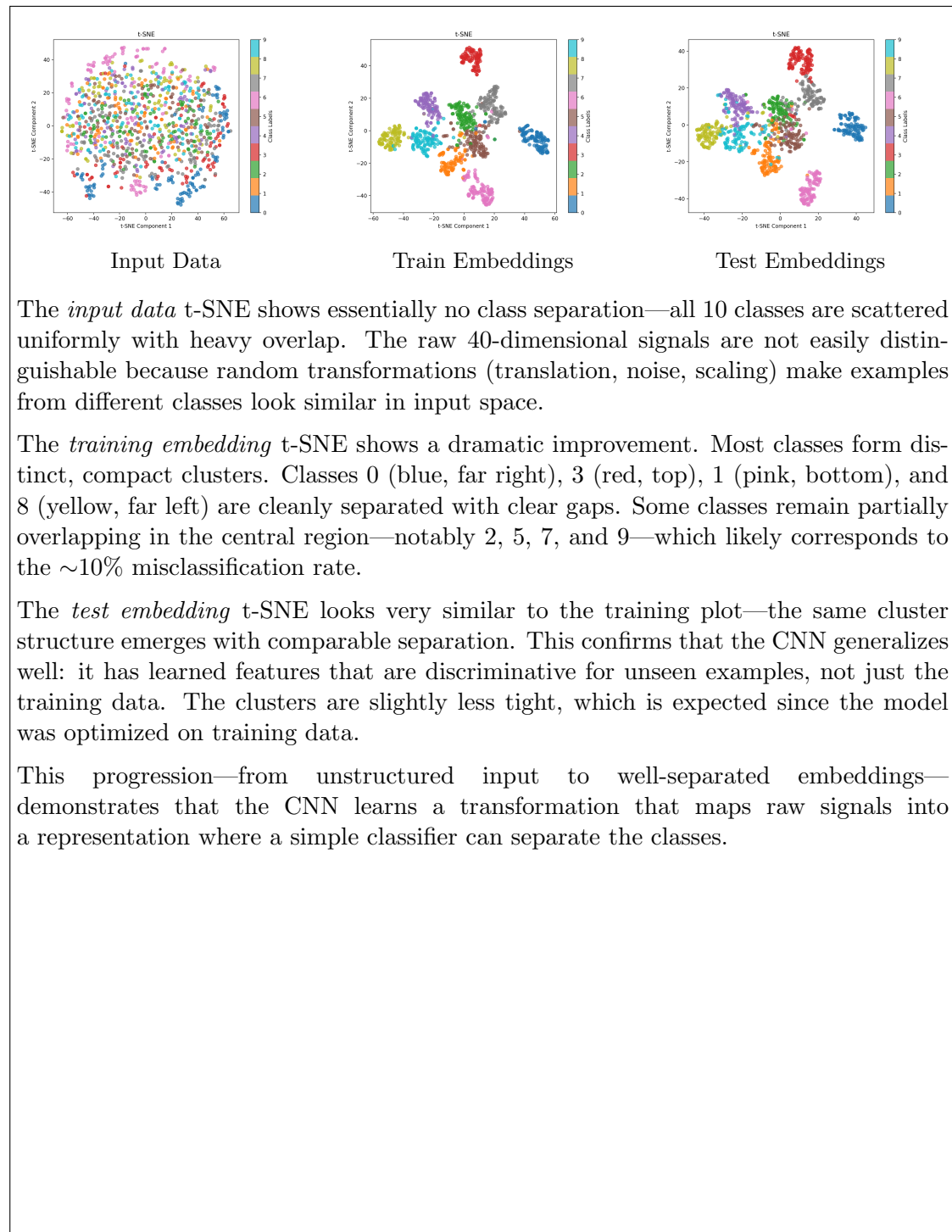
Task 3c)

Input data: Classes will be poorly separated with heavy overlap, since the raw signals are distorted by translation, noise, and scaling.

Training embeddings: Classes should form distinct, well-separated clusters, as the CNN has learned to extract discriminative features.

Test embeddings: Similar cluster structure to training embeddings if the model generalizes well (~89% accuracy), but clusters may be slightly less tight because of the generalisation error on unseen examples during training.

Task 3d)



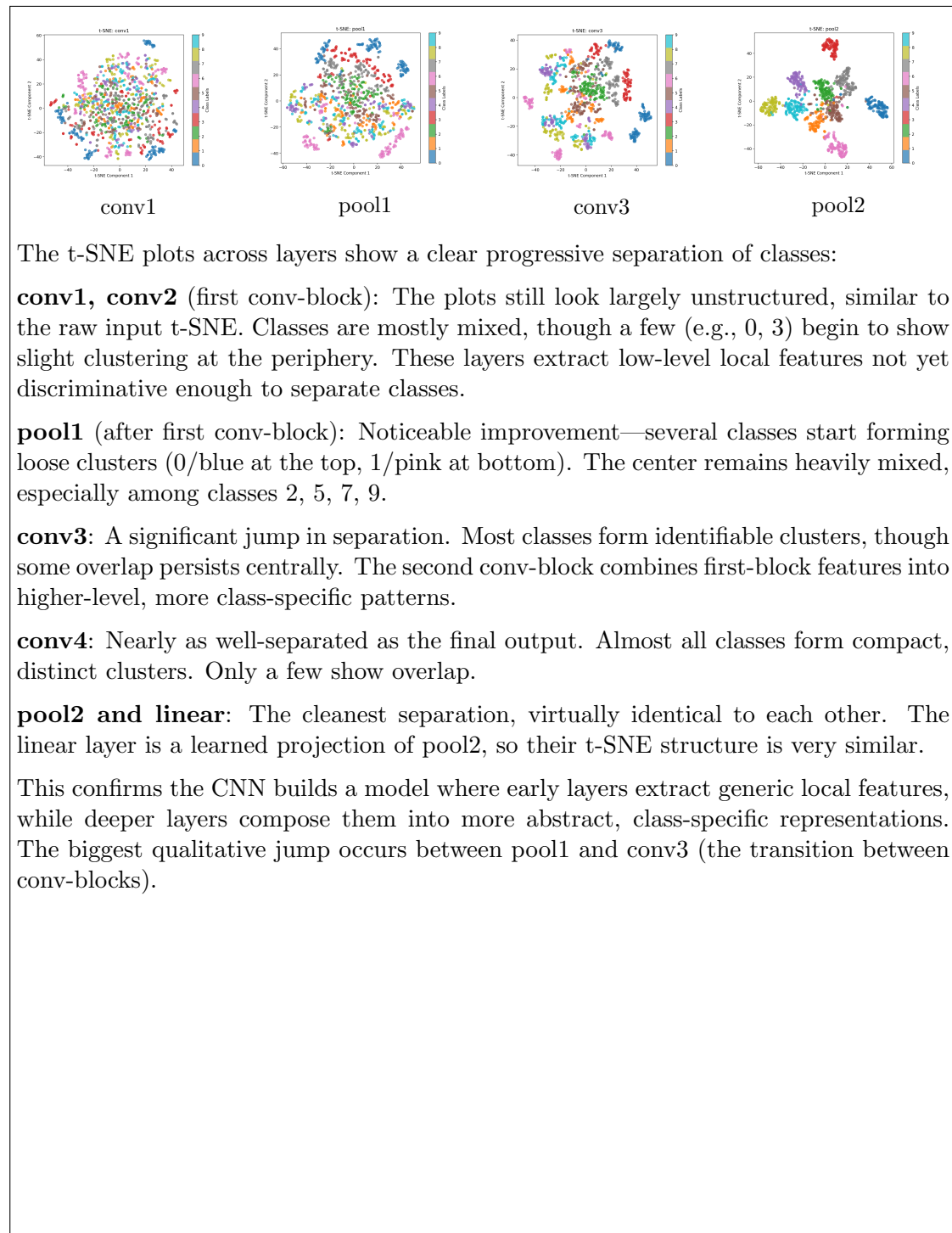
The *input data* t-SNE shows essentially no class separation—all 10 classes are scattered uniformly with heavy overlap. The raw 40-dimensional signals are not easily distinguishable because random transformations (translation, noise, scaling) make examples from different classes look similar in input space.

The *training embedding* t-SNE shows a dramatic improvement. Most classes form distinct, compact clusters. Classes 0 (blue, far right), 3 (red, top), 1 (pink, bottom), and 8 (yellow, far left) are cleanly separated with clear gaps. Some classes remain partially overlapping in the central region—notably 2, 5, 7, and 9—which likely corresponds to the $\sim 10\%$ misclassification rate.

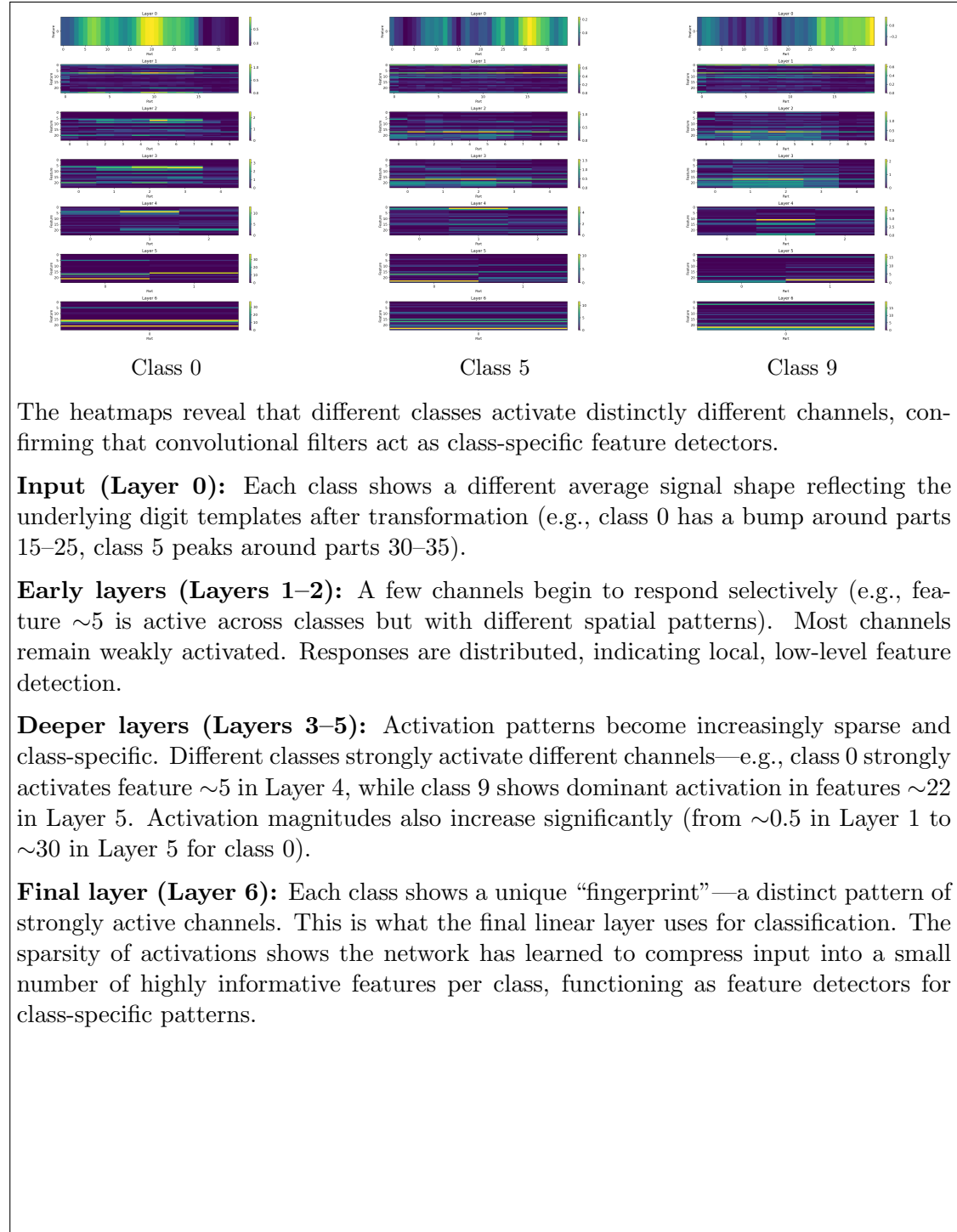
The *test embedding* t-SNE looks very similar to the training plot—the same cluster structure emerges with comparable separation. This confirms that the CNN generalizes well: it has learned features that are discriminative for unseen examples, not just the training data. The clusters are slightly less tight, which is expected since the model was optimized on training data.

This progression—from unstructured input to well-separated embeddings—demonstrates that the CNN learns a transformation that maps raw signals into a representation where a simple classifier can separate the classes.

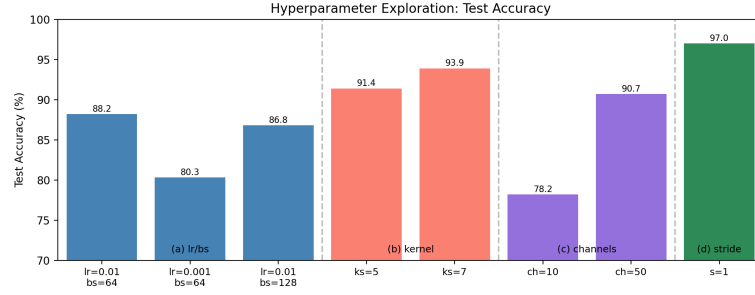
Task 3e)



Task 3f)



Task 4)



Experiment	HP	Params	Train	Val	Test
lr=0.01, bs=64 (baseline)	—	6060	99.1	88.3	88.2
lr=0.001, bs=64	lr	6060	88.1	74.6	80.3
lr=0.01, bs=128	bs	6060	100.0	87.8	86.8
kernel_size=5, pad=2	ks	9860	100.0	92.4	91.4
kernel_size=7, pad=3	ks	13660	100.0	91.9	93.9
channels=10	ch	1080	89.6	78.1	78.2
channels=50	ch	23360	99.5	92.5	90.7
stride=1	s	8310	100.0	96.3	97.0

The baseline configuration uses lr=0.01, bs=64, kernel_size=3, channels=25, stride=2, padding=1. The different configurations, if not explicitly mentioned, have the same parameter configuration.

(a) Reducing lr to 0.001 hurts performance (80.3% vs. 88.2%)—100 epochs is insufficient at this rate. Batch size 128 slightly reduces accuracy (86.8%) due to fewer updates per epoch.

(b) Larger kernels improve accuracy: ks=5 (91.4%) and ks=7 (93.9%) both outperform the baseline (88.2%). Wider receptive fields capture broader patterns, though at the cost of more parameters (13660 for ks=7).

(c) More channels help: 10ch (78.2%) → 50ch (90.7%), but gains diminish relative to the 4× parameter increase (1080 → 23360).

(d) Stride=1 achieves **97.0%** with only 8310 parameters by preserving spatial resolution. We also considered stride=3, but it reduces the spatial dimension to 0 at the final maxpool, making it infeasible.

Conclusion: Stride=1 (lr=0.01, bs=64, kernel_size=3, channels=25, padding=1) yields the best accuracy with modest parameter cost. Kernel size=7 (lr=0.01, bs=64, channels=25, stride=2, padding=1) is second best (93.9%) but requires more parameters.

Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Declaration of Used AI Tools

Task	Solved with AI?	Tool	Purpose	Useful?
1a	No			
1b	No			
2a	Yes	Claude	Cleanup writing	yes
2b	No			
2c	No			
2d	Yes	Claude	Cleanup writing	yes
2e	Yes	Claude	Cleanup writing	yes
2f	No			
2g	No			
2h	No			
2i	No			
3a	No			
3b	Yes	Claude	Code debugging assistance	yes
3c	Yes	Claude	Cleanup writing	yes
3d	No			
3e	Yes	Claude	Cleanup writing	yes
3f	Yes	Claude	Code debugging assistance	yes
4	Yes	Claude	Cleanup writing and Visualization format	yes

Unterschrift

Mannheim, den 26.04.2026



Hai Hoang Nguyen